Servlet Fundamentals

✓ What is a Servlet? Why use it in web apps?

Explanation:

A **Servlet** is a Java program that runs on a web server (like Apache Tomcat) and handles **HTTP requests and responses**. Think of it as a middleman between the client (browser) and server logic.

- When you visit a website and submit a form or click a button, your browser sends an **HTTP request**.
- The **Servlet** processes this request, executes any logic needed (e.g., reading data, querying database), and sends back an **HTTP response** (like an HTML page).

Why use Servlets?

- Servlets allow Java programs to create **dynamic web content** (pages that change based on user input or data).
- They handle all kinds of web interactions securely and efficiently.
- Java Servlets form the core of many enterprise Java web applications.

---

2. Java EE Architecture Basics

Explanation:

Java EE (now Jakarta EE) is a platform that provides a set of specifications and APIs to build enterprise-level applications.

- At its core, it follows a **client-server model**.
- **Servlets** are part of the **Web Component Layer** which handles HTTP requests.
- They usually work alongside JSPs (JavaServer Pages) and other Java EE components.

Simple Architecture Flow:

```
Browser (Client)
   ↓ sends HTTP request
Web Server (Tomcat)
   ↓ invokes Servlet
Servlet processes request → generates
response
   ↓ sends HTTP response
Browser renders response (HTML, JSON,
etc.)
```

---

3. Setting up Eclipse and Apache Tomcat

Steps:

- Download and install **Eclipse IDE for Enterprise Java and Web Developers**.
- Download and install **Apache Tomcat** server.
- Configure Tomcat in Eclipse:
    1. Go to **Servers** tab in Eclipse.
    2. Right-click → **New** → **Server**.
    3. Choose the Tomcat version you installed.

4. Point Eclipse to Tomcat installation directory.
5. Add your web project to the server.

This setup allows you to **write Java Servlets and run them inside Tomcat**, which acts as your web server.

---

4. Creating Your First Servlet with doGet() and doPost()

What are doGet() and doPost()?

- These are **methods inside the Servlet** that handle **GET** and **POST** HTTP requests, respectively.
- Every HTTP request sent by the browser has a method type. Two common ones are:
  - **GET:** Usually to fetch or request data.
  - **POST:** Usually to submit data (e.g., form submission).

Simple Servlet Example:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

   // Handles GET requests
   protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

      // Set response content type
      response.setContentType("text/html");

      // Get writer to send response
      PrintWriter out = response.getWriter();

      // Write response HTML
      out.println("<h1>Hello World!</h1>");
      out.println("<p>Current time: " + new java.util.Date() + "</p>");
   }
}
```

- When you access http://localhost:8080/yourApp/hello via browser, doGet() runs and returns HTML showing "Hello World" and the current date/time.

---

5. How HTTP Requests and Responses Work

HTTP Request

- Sent by browser/client.
- Contains:
  - Request method (GET, POST, etc.)
  - URL (Uniform Resource Locator)

- Headers (metadata)
- Parameters (query string or form data)

In Servlet, you read these via the HttpServletRequest object.

HTTP Response

- Sent by server back to client.
- Contains:
  - Status code (200 OK, 404 Not Found, etc.)
  - Headers (content-type, cookies, etc.)
  - Body (HTML, JSON, etc.)

In Servlet, you create this via the HttpServletResponse object.

Real-time Example Recap: Hello World Servlet with Current Date/Time

- Create a Servlet mapped to /hello.
- Use doGet() to respond to requests.
- Send a simple HTML response with "Hello World" and current system time.
- Deploy on Tomcat via Eclipse.
- Open URL in browser and see live response.

Extra Content:

Understand HTTP Protocol Basics

- HTTP is stateless (no memory between requests).
- Client sends requests, server sends responses.
- Web apps build logic on top of this.

Differentiate GET vs POST

Method	Usage	Parameters sent in	Data visibility	Idempotent?
GET	Retrieve data	URL (query string)	Visible in URL	Yes (safe to repeat)
POST	Submit/send data (e.g. forms)	Request body	Not visible in URL	No (may cause changes)

Introduction to HttpServletRequest and HttpServletResponse

- HttpServletRequest: Reads request info like parameters, headers, session.
- HttpServletResponse: Writes response data, sets headers and status codes.

Summary for Step 1

Concept	What It Means	Why Important
Servlet	Java program to handle web requests	Core of Java web applications

Concept	What It Means	Why Important
doGet(), doPost()	Methods to handle GET and POST requests	Different HTTP methods need different handling
HttpServletRequest	Object to get client request details	Needed to process inputs
HttpServletResponse	Object to send data back to client	Controls what client receives
HTTP Protocol Basics	Communication rules between client & server	Foundation for web communication

---

✓ What is a Web Application?

A **Web Application** is a **software program that runs on a web server** and can be accessed through a web browser over the Internet or an intranet. Unlike desktop applications (which run on your computer), web apps are accessed using **HTTP/HTTPS protocols** and do not require installation on the client machine.

---

1. Key Components of a Web Application

A web application typically has **three main layers**:

a) Client Side (Front-end)

- Runs in the **web browser**.
- Includes **HTML, CSS, JavaScript**, and front-end frameworks like **React**, **Angular**, or **Vue**.
- Responsible for **UI/UX**, collecting input, and displaying responses from the server.

Example:

- Login page form (<form> in HTML).
- Dynamic shopping cart updates with JavaScript.

---

b) Server Side (Back-end)

- Runs on a **web server** like **Apache Tomcat**, **GlassFish**, or **WildFly**.
- Contains business logic, security checks, database access, and **Servlets**.
- Handles HTTP requests from clients and sends back responses.

Example:

- A **LoginServlet** validates username/password.
- A **ShoppingCartServlet** calculates the total price.

---

c) Database Layer

- Stores persistent data like users, products, orders, etc.
- Accessed from the server using **JDBC** (Java Database Connectivity) or ORM frameworks like **Hibernate**.

Example:

- MySQL database stores user credentials.
- Servlet queries the database to validate login.

---

2. How a Web Application Works (Request-Response Cycle)

1. **User Action:** User opens a URL in a browser or submits a form.
2. **HTTP Request:** Browser sends a request to the server.
3. **Server Processing:** Servlet receives the request, executes business logic, interacts with the database.
4. **HTTP Response:** Servlet sends HTML, JSON, or XML back to the browser.
5. **Browser Rendering:** Browser displays the content dynamically.

Web Application Architecture

Explanation of Each Layer

1. **Browser (Client Layer)**
   - Users interact with HTML pages, forms, buttons.
   - Sends **HTTP requests** to the server when an action occurs.
2. **Web Server**
   - Receives HTTP requests.

    - Passes the request to the appropriate **Servlet**.
    - Handles static content like images, CSS, or JS files.
3. **Servlet (Business Logic Layer / Controller)**
    - Reads user input using HttpServletRequest.
    - Processes data or interacts with database.
    - Generates dynamic content (HTML, JSON, XML) to send back.
4. **Database Layer**
    - Stores persistent data (users, products, orders, etc.).
    - Servlet communicates via JDBC or ORM.
    - Returns query results to Servlet for processing.
5. **Response Back to Browser**
    - Servlet formats results.
    - Web server sends HTTP response back to browser.
    - Browser renders HTML/CSS/JS to display the output.

7. Role of Servlets in Web Applications

- **Servlets** are the **server-side backbone** of Java web applications.
- They handle **HTTP requests and responses**, manage sessions, and generate dynamic content.
- In a web application:
    - HTML forms collect user input (front-end)
    - Servlets process data, perform business logic, interact with databases (back-end)
    - JSP or Servlet response displays results dynamically to the user.

✓ What is Servlet API?

The **Servlet API** is a **set of Java interfaces and classes** used to develop **Java web applications**.

It provides methods to:

- Receive **HTTP requests** from the browser
- Process user data
- Send **HTTP responses** back to the browser
- Manage **sessions and cookies**

In simple words:

Servlet API allows Java programs (Servlets) to communicate with web browsers through a web server like Tomcat.

Basic flow:

Browser → HTTP Request → Servlet API → Servlet → HTTP Response → Browser

---

Why Do We Need Servlet API?

We need the **Servlet API** because it allows **Java programs (Servlets) to interact with web browsers and handle web requests**.

Without it, **Java alone cannot process HTTP requests or send responses** to browsers. The API provides **ready-made classes and methods** to make this easy.

Key Points:

1. **Process User Requests**
   - When a user fills a form or clicks a button, the browser sends data to the server.
   - HttpServletRequest from the Servlet API lets your program read this data easily.
2. **Send Responses to the Browser**
   - You can send HTML, JSON, or other data back to the browser.
   - HttpServletResponse makes this simple.
3. **Handle HTTP Methods (GET, POST, etc.)**
   - Different types of requests (fetching data, submitting forms) require different handling.
   - Servlet API provides doGet(), doPost(), etc.
4. **Manage Sessions and Cookies**
   - HTTP is stateless. To remember users, you need sessions or cookies.
   - Servlet API has HttpSession and Cookie classes for this.
5. **Build Dynamic Web Applications**
   - Instead of static HTML pages, Servlet API allows you to generate content based on user input or database queries.

Servlet API is needed to make Java programs understand web requests, respond to users, and build dynamic web applications efficiently.

Servlet API Packages

Servlet API mainly contains two packages.

javax.servlet

This package contains **basic servlet classes and interfaces**.

Important components:

Interface / Class	Description
Servlet	Base interface for all servlets
GenericServlet	Abstract class implementing Servlet
ServletRequest	Represents client request
ServletResponse	Represents server response
ServletConfig	Provides servlet configuration
ServletContext	Represents web application environment

---

javax.servlet.http

This package provides **HTTP-specific classes** used in web applications.

Important classes:

Class	Description
HttpServlet	Base class for HTTP servlets
HttpServletRequest	Handles client request data
HttpServletResponse	Sends response to client
HttpSession	Maintains session data
Cookie	Stores small data in browser

Servlet with IDE

1. Install Required Software

1. **Eclipse IDE for Enterprise Java and Web Developers**
   - Download from Eclipse Downloads.
   - Make sure you have **Java JDK installed**.
2. **Apache Tomcat 9**
   - Download from Tomcat 9 Downloads.
   - Extract it to a folder (e.g., C:\Apache\Tomcat9).

---

2. Configure Tomcat in Eclipse

1. Open Eclipse → go to **Servers** tab.
2. Right-click → **New** → **Server**.
3. Select **Tomcat v9.0 Server** → Next.
4. Browse to your **Tomcat installation folder**.
5. Add your **web project** to the server → Finish.

Now Tomcat is ready to run your Servlets inside Eclipse.

---

3. Create a Dynamic Web Project

1. **File → New → Dynamic Web Project**
2. Project name: MyFirstServlet
3. Target runtime: **Apache Tomcat 9**
4. Dynamic web module version: 3.1 (or 4.0)
5. Finish

---

4. Add a Servlet

1. Right-click **src → New → Servlet**
2. Name: HelloServlet
3. Package: com.example
4. URL mapping: /hello

Eclipse will create a **skeleton servlet** with doGet() method.

---

5. Write Servlet Code

```
package com.example;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

   protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
           throws ServletException, IOException {

      response.setContentType("text/html");

      PrintWriter out = response.getWriter();
      out.println("<h1>Hello World!</h1>");
      out.println("<p>Welcome to Servlets with Eclipse & Tomcat 9</p>");
   }
}
```

---

6. Deploy and Run Servlet

1. Right-click **project → Run As → Run on Server**
2. Choose **Tomcat 9**
3. Eclipse will start Tomcat and deploy your project
4. Open browser → type:

http://localhost:8080/MyFirstServlet/hello

You will see:

Hello World!
Welcome to Servlets with Eclipse & Tomcat 9

---

1. What is a Servlet Request?

When a user **visits a website or submits a form**, their **browser sends information to the server**.

- This information is called a **request**.
- In Java Servlets, the **request is handled using the HttpServletRequest object**.

Think of it like:

"The browser is asking the server something."
The HttpServletRequest object **lets your servlet read what the browser asked for**.

---

2. Why Do We Need Servlet Request?

We need it because:

1. To **read data from the user**, like a username or password.
2. To **know what kind of request** it is (GET or POST).
3. To **access session or cookies**.
4. To **read headers** like browser type.

Without HttpServletRequest, the servlet **cannot know what the user sent**.

---

3. Simple Real-Time Example

Imagine you have a **login page**:

HTML Form:

```
<form action="LoginServlet" method="post">
  Username: <input type="text" name="username"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
```

What happens when user clicks "Login"?

- Browser sends **username and password** to the server.
- HttpServletRequest lets your servlet **read this data**.

---

4. Servlet Code to Read Request Data

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;
```

```
@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request,
                         HttpServletResponse response)
           throws ServletException, IOException {

      // Read data from browser using Servlet Request
      String username = request.getParameter("username");
      String password = request.getParameter("password");

      // Send response back to browser
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      if(username.equals("admin") && password.equals("1234")) {
         out.println("<h2>Login Successful</h2>");
      } else {
         out.println("<h2>Login Failed</h2>");
      }
   }
}
```

HTML Code (login.html)

```
  <!DOCTYPE html>
<html>
<head>
<title>Login Form</title>
</head>

<body>

<h2>Login Page</h2>

<form action="LoginServlet" method="post">

<label>Username:</label>
<input type="text" name="username" required>

<br><br>

<label>Password:</label>
<input type="password" name="password" required>

<br><br>

<input type="submit" value="Login">

</form>

</body>
</html>
```

✅ **Explanation:**

- request.getParameter("username") → reads the value typed by the user.
- request.getParameter("password") → reads the password.
- response.getWriter() → sends response back to browser.

---

5. Key Points to Remember

Feature	Purpose	Example
getParameter()	Read form or URL data	request.getParameter("username")
getMethod()	Know request type (GET/POST)	request.getMethod()
getHeader()	Read browser info	request.getHeader("User-Agent")
getSession()	Access session	request.getSession()
getCookies()	Read cookies	request.getCookies()

- **Request = What the user sends**
- **Response = What the server sends back**

Servlet Collaboration

Servlet Collaboration means **one servlet working together with another servlet, JSP, or HTML page to complete a task**.

In real web applications, **one servlet usually cannot handle the entire process alone**.
So servlets **share work and pass the request to other resources**.

Simple Definition

Servlet Collaboration is the process where multiple web resources cooperate to handle a single client request.

Why Do We Need Servlet Collaboration?

Large applications have **multiple layers**:

- User interface
- Business logic
- Data processing

If one servlet handles everything, the code becomes **complex and difficult to maintain**.

Real-Time Example (Login System)

User opens login page and enters credentials.

```
Browser
   ↓
LoginServlet
   ↓
Check username/password
   ↓
Forward to Dashboard.jsp
```

If login fails:

```
LoginServlet
   ↓
Forward back to login.html
```

Methods of Servlet Collaboration

There are **two main ways**:

1. RequestDispatcher
2. SendRedirect

RequestDispatcher (Server-Side Collaboration)

The **RequestDispatcher** interface is used to **forward a request from one servlet to another resource**.

The resource can be:

- Another Servlet
- JSP page
- HTML file

Important Feature

The request is processed **inside the server**.
The browser **does not know about the forwarding**.

The URL **does not change**.

Servlet Collaboration Example Program

HTML File

```
<!DOCTYPE html>
<html>
<head>
<title>Servlet Collaboration</title>
</head>
```

```
<body>

<h2>Enter Your Name</h2>

<form action="FirstServlet" method="post">

<input type="text" name="username">

<br><br>

<input type="submit" value="Submit">

</form>

</body>
</html>
```

First Servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/FirstServlet")

public class FirstServlet extends HttpServlet {

protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

String name = request.getParameter("username");

request.setAttribute("user", name);

RequestDispatcher rd =
request.getRequestDispatcher("SecondServlet");

rd.forward(request, response);

}
}
```

Second Servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/SecondServlet")

public class SecondServlet extends HttpServlet {

protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

response.setContentType("text/html");

PrintWriter out = response.getWriter();

String name = (String) request.getAttribute("user");

out.println("<h2>Hello " + name + "</h2>");

}
}
```

What is CRUD

CRUD stands for:

Operation	Meaning
C	Create
R	Read
U	Update
D	Delete

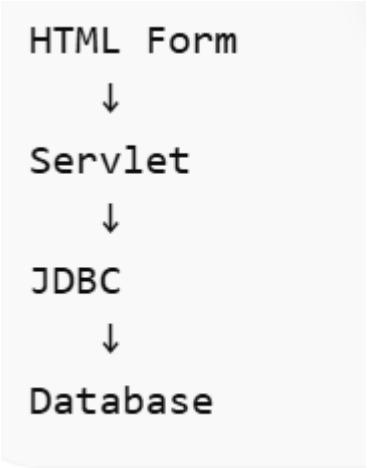
CRUD operations are used to **manage data in a database**.

Example: **Student Management System**

- Create → Add student
- Read → View student
- Update → Edit student
- Delete → Remove student

Architecture of CRUD in Servlets

Typical flow:



Steps:

1. User enters data in HTML form
2. Servlet receives request
3. Servlet connects to database using JDBC
4. SQL query executes
5. Response sent to browser

Step 1: SQL – Create Database and Table

First explain the **database structure**.

```
CREATE DATABASE studentdb;

USE studentdb;

CREATE TABLE student(
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(50),
email VARCHAR(50),
age INT
);
```

Explanation:

- CREATE DATABASE → creates database
- USE → selects database
- CREATE TABLE → creates student table
- AUTO_INCREMENT → id automatically increases

Step 2: HTML Form (Insert Data from Web Page)

After database creation, create the **web page to collect user input**.

```
<!DOCTYPE html>

<html>

<head>

<title>Add Student</title>

</head>

<body>

<h2>Student Registration</h2>

<form action="InsertServlet" method="post">

Name:

<input type="text" name="name">

<br><br>

Email:

<input type="text" name="email">

<br><br>

Age:

<input type="text" name="age">

<br><br>

<input type="submit" value="Insert">

</form>

</body>

</html>
```

Explanation:

- action="InsertServlet" → sends data to servlet
- method="post" → securely sends data
- Input fields collect user data

Step 3: Servlet Code (Insert Data into Database)

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;
@WebServlet("/InsertServlet")
public class InsertServlet extends HttpServlet {
protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
String name = request.getParameter("name");
String email = request.getParameter("email");
int age = Integer.parseInt(request.getParameter("age"));
try{
Class.forName("com.mysql.cj.jdbc.Driver");
Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/studentdb",
"root",
"password");
PreparedStatement ps = con.prepareStatement(
"insert into student(name,email,age) values(?,?,?)");
ps.setString(1,name);
ps.setString(2,email);
ps.setInt(3,age);
ps.executeUpdate();
response.getWriter().println("Student Added Successfully");
```

```
}catch(Exception e){
System.out.println(e);
}
}
}
```

Explanation:

- getParameter() → receives form data
- JDBC Driver → connects Java to database
- PreparedStatement → executes SQL insert query
- executeUpdate() → inserts record